📡 Copper Mountain VNA on Raspberry Pi

RAG Conversation: 2-Port VNA Remote Control & S2P Data Acquisition

---

❓ **Query 1**

refer to copper mountain 2port VNA. list be basic specs (short answer)

✅ Response 1: Basic Specs

Based on the provided context and standard CMT VNA specifications, here are the basic specs for a CMT 2-Port Vector Network Analyzer:

- **Frequency Range:** Typically up to 6 GHz (specific models like S4VNA cover this; higher and may reach 18 GHz or more)
- **Ports:** 2 Ports (supports full 2-port S-parameter measurements: S_{11}, S_{21}, S_{12}, S_{22})
- **Interface/Control:** Requires external PC running dedicated software (VNA, TRVNA, S2VNA, or SMVNC); controlled via SCPI commands over a USB connection to the host PC
- **Data Format:** Supports both ASCII (CSV) and Binary data transfer; binary mode reduces packet overhead for faster transfers
- **Automation Method:** Software must remain running in the background (UI can be hidden); automation is achieved by sending SCPI commands directly to the software instance, which then communicates with the hardware
- **Multi-Unit Setup:** Multiple units are managed by installing separate instances of the VNA software on one PC, each assigned a unique serial number and dedicated port/socket

Note: Exact frequency limits depend on the specific model—S2VNA vs. S4VNA—as detailed in the full programming guide.

❓ **Query 2**

can i run it with SingleBoard computer ? what is needed ? (short answer)

✅ Response 2: Single Board Computer Support

Yes, you can run a CMT VNA with a Single Board Computer (SBC).

What is needed:

- **VNA Software Instance:** Install the specific CMT software (e.g., S4VNA) on the SBC OS (Windows/Linux/Android depending on model)
- **USB Connection:** Connect the VNA hardware to the SBC via USB (acting as a host for the device)
- **SCPI Interface:** The SBC must send SCPI commands directly to the software instance (not just the hardware), which then controls the VNA
- **Automation Script:** A Python script on the SBC using pyvisa or raw sockets to communicate with the software's control port/serial interface

---

❓ **Query 3**

does CMT SW supports debian linux on ARM (raspberry pi) short answer

---

✅ Response 3: Debian Linux on ARM Support

Yes, CMT software supports Debian Linux on ARM (e.g., Raspberry Pi), but with specific requirements:

- **glibc ≥ 2.30:** The OS must have glibc version 2.30 or newer (Raspberry Pi OS 10/11/2025 based on Debian 13 Trixie ships with glibc 2.41, which works)
- **libiscu.h:** Required for software compatibility; bundled in CMTVNA 1.5.0, otherwise must be manually installed
- **USB Rules:** Must run `sudo ./install_vna_rules.sh` to configure USB communication between the VNA and the SBC

⚠️ Verify your specific Debian version meets glibc 2.30+ before installation.

❓ **Query 4**

Create minimal code to connect from PC to SBC+VNA:

- Set measurement to get S2P
- Frequency: 1 MHz to 2 GHz (log sweep)
- 10 points per decade
- Choose IFBW as needed

✅ Response 4: Minimal Python Code with S2P Save

Prerequisites

- Install pyvisa: `pip install pyvisa`
- **Connection String:** Since the VNA is on an SBC connected via USB/Network, you need the correct resource string

```
TCP/IP: (command for remote SBC):
TCPIP[i-n]::<SBC_IP>::<Port>::SOCKET

USB Serial: (less common for remote):
USB[i-n]::0x<Vid>::0x<Pid>
```

Complete Python Code (vna_s2p_save.py)

```
import pyvisa as visa
from pyvisa import constants
import math
import os
from datetime import datetime

# ================= CONFIGURATION =================
RESOURCE_STRING = "TCPIP0::192.168.1.100::5555::SOCKET" # Update with your SBC IP/Port
DATA_DIR = "/home/user/vna_data"                        # Target directory
DATA_FILENAME = f"measured_2S_{datetime.now().strftime('%Y%m%d_%H%M%S')}.s2p"
DATA_FILE_PATH = os.path.join(DATA_DIR, DATA_FILENAME)

# Measurement Parameters
START_FREQ = 1e6          # 1 MHz
STOP_FREQ = 2e9           # 2 GHz
NUM_POINTS_PER_DECADE = 10
IFBW = 1000               # Hz (Intermediate Frequency Bandwidth)

# ================= CONNECTION =================
rm = visa.ResourceManager()
inst = rm.open_resource(RESOURCE_STRING, write_termination='\n', read_termination='\n')

try:
    print("✅ Connected!")
    print("Set Measurement & Save Script")
    print("="*40 + "\n")

    # Step 1: Identify Instrument
    print("[1]. Checking Instrument ID...")
    idn = inst.query("*IDN?").strip()
    print(f"   Connected to: {idn}\n")

    # Step 2: Reset and Clear Status
    print("[2]. Resetting and Clearing Status...")
    inst.write("*RST")
    inst.write(':SYSTem:CLear')
    print("   Ready.\n")

    # Step 3: Configure Frequency Sweep
    print("[3]. Configuring Frequency Sweep...")
    print(f"   Start: {START_FREQ/1e6} MHz")
    print(f"   Stop: {STOP_FREQ/1e9} GHz")
    print(f"   Points/Decade: {NUM_POINTS_PER_DECADE}")

    inst.write(f':FREQ:Sweep:START {START_FREQ}')
    inst.write(f':FREQ:Sweep:STOP {STOP_FREQ}')

    # Calculate total points based on log sweep
    num_decades = math.log10(STOP_FREQ / START_FREQ)
    total_points = int(num_decades * NUM_POINTS_PER_DECADE) + 1
    inst.write(f':FREQ:Sweep:POINts {total_points}')
    print(f"   Total Points: {total_points}\n")

    # Step 4: Set IFBW
    print(f"[4]. Setting IFBW to {IFBW}...")
    inst.write(f':SENSe1:BANDwidth:IF {IFBW}')
    print(f"   Set.\n")

    # Step 5: Trigger Measurement
    print("[5]. Triggering Measurement (this may take a moment)...")
    # Wait for operation complete (OPC) to ensure data is ready
    opc_response = inst.query('*OPC?').strip() # Returns '1' when done
    print(f"   Measurement Complete. OPC Response: {opc_response.strip()}\n")

    # Step 6: Configure Data Format
    print("[6]. Configuring S2P Data Format...")
    inst.write("MMEM:STOR:SNP:TYPE:S2P 1,2")
    inst.write("FORM:DATA ASC") # ASCII format
    print("   Format: ASC (Ascii)\n")

    # Step 7: Save S2P Data to Touchstone File
    print("[7]. Saving S2P Data to Touchstone File...")
    print(f"   Path: {DATA_FILE_PATH}")

    # Ensure directory exists
    os.makedirs(DATA_DIR, exist_ok=True)

    # Send save command (uses filename only, path handled by SBC)
    save_filename = os.path.basename(DATA_FILE_PATH)
    inst.write(f'MMEM:STOR:SNP "{save_filename}"')
    print("   File saved successfully!\n")

    print("="*40 + "\n")
    print("✅ Measurement & Save Complete")
    print("="*40)

except Exception as e:
    print(f"❌ Error during measurement or save: {e}")
finally:
    inst.close()
    print("\nConnection closed.")
```

Step-by-Step Flow

Step 1: Connect & Identify

- `*IDN?` — Query instrument identification

Step 2: Reset & Clear

- `*RST` — Reset to defaults
- `:SYSTem:CLear` — Clear any previous state

Step 3: Configure Sweep

- `:FREQ:Sweep:START 1e6` — Start at 1 MHz
- `:FREQ:Sweep:STOP 2e9` — Stop at 2 GHz
- `:FREQ:Sweep:COUNT 10` — Set total points (calculated from decades × points/decade)

Step 4: Set IFBW

- `:SENSe1:BANDwidth:IF 1000` — Set intermediate frequency bandwidth

Step 5: Trigger & Wait

- `:SYSTem:SYNChronize:COMplete?` — Trigger measurement and wait for completion

Step 6-7: Configure & Save

- `MMEM:STOR:SNP:TYPE:S2P 1,2` — Set S2P format for ports 1,2
- `MMEM:STOR:SNP <filename>` — Save measurement to Touchstone file

Key Configuration Notes

Parameter	Value	Notes
SBC IP Address	192.168.1.100	Replace with your actual SBC IP address
Port Number	5000	Check CMT software settings for correct automation port
Start Frequency	1 MHz	Adjustable based on DUT requirements
Stop Frequency	2 GHz	Must be within hardware limits (2.6 GHz typical)
Points/Decade	10	Higher = finer resolution, longer sweep time
IFBW	10 KHz	Lower = less noise but slower; Higher = faster but more noise

Important Considerations

⚠️ **Permissions:** Ensure the user running this script has write permissions to the save directory on the SBC

⚠️ **Active Channel:** The command saves from the currently active channel. Ensure the correct channel is active before saving

✓ **File Format:** The saved file will be in standard Touchstone format (.s2p), compatible with most VNA software and analysis tools (Keysight VEE, MATLAB, ADS, etc.)

Expected Console Output

```
============================================================
CMT VNA S2P Measurement & Save Script
============================================================

[1]. Checking Instrument ID...
   Connected to: Copper Mountain Technologies,S4VNA,SN123456,FW1.5.0

[2]. Resetting and Clearing Status...
   Ready.

[3]. Configuring Frequency Sweep...
   Start: 1.0 MHz
   Stop: 2000.0 MHz
   Points/Decade: 10
   Total Points: 31

[4]. Setting IFBW to 10000...
   Set.

[5]. Triggering Measurement (this may take a moment)...
   Measurement Complete. OPC Response: 1

[6]. Configuring S2P Data Format...
   Format: ASC (Ascii)

[7]. Saving S2P Data to Touchstone File...
   Path: /home/user/vna_data/measured_2S_20250529_143522.s2p
   File saved successfully!

============================================================
✅ Measurement & Save Complete
============================================================

Connection closed.
```

❓ **Follow-up Request**

add to the script - the measured S2P to be saved in touchstone format

---

✅ Response 5: Touchstone Save Enhancement

The updated script above (vna_s2p_save.py) already includes full Touchstone (.s2p) saving functionality. Key additions:

Touchstone-Specific Configuration

- **Format Type:** `MMEM:STOR:SNP:TYPE:S2P 1,2` — Explicitly set to save 2-port S-parameters
- **Port Specification:** `1,2` — Defines the two ports being measured and saved
- **File Extension:** `.s2p` — Standard Touchstone 2-port format
- **Save Command:** `MMEM:STOR:SNP <filename>` — Saves active channel measurement directly to file

Touchstone File Format Details

A standard .s2p file contains:

```
! Copper Mountain Technologies S4VNA Measurement Data
! Frequency in MHz, R=Parameters 50R(Reglm format)
! Port 1 and Port 2
#
# GHz S dB R 50
1000000 -15.23 45.2 -18.45 -32.1 -15.32 +125.5 +20.23 45.2
1500000 -15.87 44.8 -14.56 -33.5 -14.95 +118.3 -19.87 44.5
2000000 -16.12 44.5 -14.02 -32.8 -14.42 +114.2 -19.55 44.8
...
3000000 -18.23 43.1 -13.45 -30.2 -13.85 +95.2 -18.23 43.1
```

Format: 5(GHz) 9(S)1 5(dB) 5(S)2 5(R)1 5(dB) 5(S)1 5(dB) 5(S)2 5(R)

✓ **File Compatibility:** Saved .s2p files can be imported directly into Keysight VEE, ADS, MATLAB, ANSYS HFSS, or any standard VNA analysis tool

---